The AI-EDGE program has been a very fruitful endeavor for me. Before starting the program, I knew nothing about artificial intelligence outside what is known in the mainstream. This was despite being a computer science student. After attending the program, not only do I have a much deeper understanding of artificial intelligence, neural networks, data models, and the underlying math behind it all, I also have found a new area of academia I would like to explore further in the future: data science and statistical analysis.

One great example of where statistics meets computer science is gradient descent, one of the fundamental building blocks of modern AI. In the opening lectures, we discussed how we can train models to minimize or maximize the value of a certain function in order to reach a desired output. Based on fundamental knowledge of computing, many problems found in the world of computing can be derived from optimization problems. In fact, in the paper I researched, Google's AlphaGo, gradient ascent was used to maximize positional strength in Go via high level tree search and move selection. AlphaGo also leveraged recent advancements in neural networks to great benefit in it's implementation of Monte Carlo Tree Search.

Neural networks was another topic we covered during our lectures. The main point which I found interesting from our time covering the topics was how to teach a neural network context. For example, in language processing models, the context in which a word is spoken is almost as important as the word itself. However, a naive solution to the problem using a neural network would be a static embedding solution. This losses a lot of the nuance and complexity of spoken words. We can alleviate this by creating a new vector which can hold the context of a given word. Within this vector, we need to pay attention to certain words in order to match the current word to the correct context, and this can be easily achieved by taking a weighted sum of previous tokens, using a function like softmax.

Softmax is one of many functions which takes a series of vectors of numbers and then converts them into probability distributions. In the paper, another function which was used was minimax. While it sounds similar, softmax and minimax are actually quite distinct. Minimax sees the majority of it's use in games and game theory. The idea is that, given some mathematical evaluation of a game, choose the maximizing move for the player. In Go, for example, minimax is used to allow AlphaGo to select actions from a set of node in the decision tree. Minimax, along with many other functions, search all possible options, and then return the maximizing option for AlphaGo. This is a very different approach compared to the normalizing functions of softmax, although both are used in AlphaGo.

Afterwards, we moved on to image generation and manipulation using our knowledge. While I did not fully understand this section, I did gain useful insight into some differing areas of machine learning. For example, the feed-forward neural network for token classification for images was a interesting concept. In this implementation, we are attempting to minimize evidence lower bound (ELBO) in order to have the best chance at creating an accurate image based on some input. This is done by encoding an input into individual tokens, and then decoding the tokens into a new image. A second way of doing this would be using a generative adversarial network (GAN). GANs are quite unique in the sense that it is essentially having the model compete against itself to create a highly accurate image. It does this by generating an image, and then having a discriminator model determine the probability of if the image is real. After several rounds of this back and fourth, a highly accurate image is constructed. While GANs are quite intriguing, they are also comparatively unstable, needing levels of knowledge in order to prevent mode collapse and other such pitfalls.

The topic of which I had the most prior knowledge in before attending the program was Professor Jim Kurose's presentation on AI application in wireless networking. I had worked at a broadband company for 2 years in the technical support division, and while I mostly handled small scale things, I had also encountered many of the high level problems he was describing. I also have a background in networking fundamentals. I thoroughly enjoyed listening about how AI can be used to do things like seamless 5G handover and packet scheduling. While I haven't found the time for it quite yet, I do hope to read some of the supplementary materials he provided in his presentation.

For my paper, I selected "Mastering the Game of Go with Deep Neural Networks and Tree Search". The article goes into the methodology and implementation of AlphaGo, a CNN model developed to play expert level Go by leveraging Monte Carlo Tree Search with new neural network techniques to efficiently reduce the search space of the decision tree. Before I explain my technical findings, some context given within the paper may be needed.

The game of Go is played on a 16 x 16 board, where two players (white and black) take turns placing stones on the board until one concedes, there are no legal moves remaining, or both opponents agree to conclude the game. While I personally have played chess at an average level, I have never played Go before. However, even though I am unfamiliar with the specifics of the game of Go, I do understand the fundamental mechanics of what the paper describes as Markov games. In Markov games, the game can be described as a grid of base units by width units (16 x 16 in this case) with some current state **s**. The state **s** can be any legal arrangement of pieces on the board, including having no pieces at all. Each turn, players may make an action **a**.

The action **a** alters the state on the board. These actions are committed until the game reaches a conclusion at turn **T**. For games described in this manner, there is an optimal value function, in the paper described as v*(s), which “determines the outcome from state s following perfect play by both players”.

The challenge is obvious: create a program which can recreate v*(s) as accurately as possible. This is easier said than done. Prior research stated that developing AI which could play Go at a high level like this was quite some time away, as far off as a decade or more. This is due to the models being unable to effectively truncate the search space of the decision tree. While some groups had experimented using Monte Carlo Tree Searches, a methodology which the AlphaGo team was able to leverage to an astounding degree, an overreliance on the rollout policy led to decreased performance from the machine. While Monte Carlo Tree Search (MCTS) is a major portion of how AlphaGo achieves it’s near inhuman level of performance, one of the key innovations is how the researchers chose to implement MCTS, as a part of a whole, and also how the use board state analysis via a value network to effectively guide the AI towards optimal decision making. To quote the paper, “AlphaGo combines the policy and value networks in an MCTS algorithm that selects actions by lookahead search”.

However, before the AI is ready to play any actual games, it must be trained to be able to do so. In fact, researching the training regime of AlphaGo was almost as interesting as understanding the machine itself. In order to get the neural networks tuned to appropriate levels, the team used a three step approach. First, a 13-layer policy network is trained using supervised learning (SL). The network is fed 30 million different board states from the KGS Go Server, and is given the task to maximize the chance of selecting an expert level move from the given state. This is done using stochastic gradient ascent. When training had concluded, the network was able to accurately predict the expert move 55.7% of the time using only the given board state and move history. This was a 7% increase from prior work at the time. Concurrently, a faster rollout policy is also trained on similar data. This policy network is much faster, selecting actions in just 2μs compared to 3 ms, however only has an accuracy of 24.2%. This degradation in accuracy, however, is acceptable for it’s use case.

After both the SL policy network and the fast-rollout policy network are finished, the training moves on to gradient reinforcement learning (RL). A new policy network, identical to the SL network, is created. This new network then plays games against randomly selected previous iterations of the SL policy network. The randomization helps stabilize training patterns. After arbitrary amounts of time, the weights of the RL policy network are updated using stochastic gradient ascent, maximizing expected outcome.

Lastly, the value network is trained. The value network is arguably the most important part of AlphaGo. It's goal is, given a board state **s**, to predict the outcome of the game assuming both players play according to a given policy **p**. In a perfect situation, the position would be evaluated using v*(s), however in practical uses, this value function is estimated using the value network. The value network is similar to the policy networks, besides the output. The network is trained on state-outcome pairs, and using gradient decent, is trained to minimize MSE between predicted value, and true outcome. One of the ways the paper achieves this is by feeding the value network data from the games of the RL policy network, and then selecting uncorrelated positions and having the value network randomly sample moves from the game until the game ended. Now that both the policy and value networks have been trained, the AI is essentially "game ready". To see this, lets look at what a turn of Go looks like for the AI.

For any given turn, the AI has 4 distinct stages in it's search algorithm. First is the selection stage. Imagine a tree, with a root node **s** being the current state of the game. The algorithm will traverse the decision tree in a way which maximizes the AI's position until a leaf node is reached after a specified amount of time. This process is essentially the first of two rollout procedures. Next, the AI will evaluate the leaf node. To do this, the leaf node's board state is added to a queue in order to be evaluated by the value network. Once the initial state has been evaluated, if the value network deems the state to have potential, the quick rollout policy will essentially simulate a somewhat optimal game from the position until termination. Once the rollout has terminated, the overall winner is calculated. Then, two backwards passes are conducted through the tree, to the initial leaf node. The first pass updates the nodes with the actual evaluations and outcomes for their specific game state. The second pass allows the value network to more closely assess the individual positions for any given node. A final subsystem allows for more focused analysis of certain outcomes; the expansion system. For any given node, if the visit count exceeds a certain value, it will be more closely examined by both the value and policy networks. This is done by adding the node into a separate queue for asynchronous computation, with accompanying algorithms to ensure that the number of tasks added is scaled appropriately to available compute power. The expansion policy is ultimately the deciding factor in how the AI decides what actions to take. This is because the most visited move is ultimately the one selected. The reasoning given is to avoid outliers which can occur when choosing to maximize action value.

To actually bring all of this ingenuity and engineering "to life" it was also important that the system was able to be run asynchronous across multiple CPUs and GPUs. This is so all the various systems have proper resources to compute what is needed.

Ultimately, AlphaGo used “40 search threads, 48 CPUs, and 8 GPUs”. There also exists a distributed version of AlphaGo, although it was not used in the actual matches.

As someone who enjoys games, the paper was quite an interesting read for me. Seeing the cutting-edge of decision making AI, how they actually operate, and the math which controls them, opened my eyes to the potential of statistics and data science in areas of computer science outside of big data. For example, a popular show on TV *BattleBots* has teams building robots to fight in an arena. What’s to say the future of entertainment couldn’t be the same, but with AI? However, to a more technical degree, it was intriguing to see the specifics of the rollout policy and MCTS. While I was vaguely familiar with MCTS, I had not taken the time to truly understand and appreciate the methodology until now.

My hope is to take what I have learned from my time here at the AI-EDGE program and apply it to developing my own AI as a portfolio piece. While it’s scope will most likely be rudimentary, I think it will be a fun challenge, and one that I will be able to confidently undertake now that I understand much of the foundational knowledge of machine learning. Furthermore, my dive into the subject of AlphaGo has left me with many good resources on the topic which I can reference in the future.

I would like to take this final paragraph thank all the faculty who put together this program. It has been a very enlightening experience for me, which I feel has given me a better understanding of what I can expect if I choose to pursue a higher education. I have been provided with a firm foundation, upon which I can build into a variety of skills I can use in the future. Thank you.